

Mockito

- Mockito is a popular mocking framework for Java that is widely used for unit/component testing.
- It integrates seamlessly with Java Spring Boot and JUnit 5 to facilitate the creation of mock objects and simplifies the testing process by isolating dependencies.
- It enables the creation of "mock" objects, which are simulated versions of real dependencies, allowing for isolated testing of individual components.

Mockito Concepts

1. Set the expectations

Expect

When()
Then Return()

2. Use the object being tested

Use

Test your Component

3. Verify Expectations and Results

Verify

Verify Result

- Expect: Set up the behavior of mock objects (using when and thenReturn).
- Use: Execute the component under test and use the mock objects during the process.
- Verify: Check that the desired interactions occurred and validate the results.

```
126 @DisplayName("3. Process Order with Verification")  Araf Karsh Hamid *
127 @Order(3)
128 @Test
129   void testPlaceOrder() {
130     // Given Order is Ready
131     PaymentStatus paymentAccepted = OrderMock.paymentAccepted(order);
132
133     // When: Set up the behavior of mock objects
134     when(orderRepo.saveOrder(order)).thenReturn(order);
135     when(paymentService.processPayments(order.getPaymentDetails()))
136         .thenReturn(paymentAccepted);
137
138     // Execute the Component under the Test
139     orderService.processOrder(order);
140
141     // Then
142     assertNotNull(order);
143     assertNotNull(order.getPaymentDetails());
144     assertNotNull(order.getOrderItems());
145
146     // Verify: Check that the desired interactions occurred and validate the results.
147     verify(orderRepo).saveOrder(order); // Ensure the order was saved
148     verify(paymentService).processPayments(any()); // Ensure payment was processed
149 }
```

Key Features of Mockito

- **Integration with JUnit 5:** Works seamlessly with JUnit 5 annotations and lifecycle methods.
- **Support for Annotations:** Simplifies setup and reduces boilerplate code using annotations like **@Mock**, **@Spy**, **@InjectMocks**, etc.
- **Mocking Dependencies:** Create mock objects for dependencies, allowing you to isolate the system under test (SUT) without relying on actual implementations.
- **Spy:** Partial mocking for real objects, allowing some methods to be stubbed while others behave as normal.
- **Stubbing:** Predefine the **behaviour of methods on mocked objects** (e.g., return specific values when certain methods are called).
- **Behaviour Verification:** Verify if a particular method was called on a mock object with specific arguments.
- **Argument Matchers:** Use **flexible matchers (like any(), eq(), etc.)** to match arguments while verifying or stubbing.
- **Behaviour Verification Order:** Ensure methods are called in a specific order using InOrder.
- **Exception Handling:** Simulate exception-throwing scenarios to test error handling.
- **Mock Static Methods** (with mockito-inline or third-party libraries): Support for mocking static methods.

Key Annotations

- `@ExtendWith(MockitoExtension.class)`:
 - A JUnit 5 extension that enables Mockito annotations (`@Mock`, `@InjectMocks`, etc.) and simplifies test setup.

```
@ExtendWith(MockitoExtension.class)
```

```
public class MyTest {  
    // Mock and InjectMocks annotations work seamlessly here  
}
```

- `@Mock`: Creates a mock instance of a class or interface.

```
@Mock
```

```
private PaymentService paymentService;
```

- `@Spy`: Creates a partial mock, allowing some real methods to be called while others can be stubbed.

```
@Spy
```

```
private OrderService orderService;
```

- **@Captor:** Captures arguments passed to mocked methods for verification.

```
@Captor  
private ArgumentCaptor<String> stringCaptor;
```

- **@InjectMocks:** Automatically injects mocks into the object under test.
 - Dependencies marked with @Mock or @Spy are injected into the class being tested.

```
@InjectMocks  
private WarehouseService warehouseService;
```

Annotation – @Mock, @InjectMock

Scenario

Create Mocks for the following

1. Repository Service
2. Payment Service

Annotations

- @Mock
- @InjectMocks

Test Cases

1. Given a Valid Order
2. Test for Payment Accepted
3. Test for Payment Declined

```
56 @TestMethodOrder(OrderAnnotation.class)
57 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
58 @ExtendWith(TestTimeExtension.class)
59 @ExtendWith(MockitoExtension.class)
60 public class OrderServiceTest {
61
62     static final Logger log = getLogger(lookup().lookupClass()); no usages
63
64     private OrderEntity order; 20 usages
65     private int counter = 1; 3 usages
66
67     @Mock 4 usages
68     OrderRepository orderRepo;
69
70     @Mock 4 usages
71     PaymentService paymentService;
72
73     // Order Service needs OrderRepository and PaymentService
74     // 1. Mocks are created for these two (OrderRepository, PaymentService) services
75     // 2. Inject this mocks into OrderServiceImpl
76     @InjectMocks 3 usages
77     OrderServiceImpl orderService;
78
79     /**
80      * If the @TestInstance(TestInstance.Lifecycle.PER_CLASS)
81      * is available then the method need not be static
82      */
83     @BeforeAll 1 Araf Karsh Hamid
84     public void setupAll() {
85         System.out.println("== Order Service Mock Suite Execution Started...");
86     }
87
88     @BeforeEach 1 Araf Karsh Hamid
89     public void setup() {
90         System.out.println(counter+" Create Order, PaymentStatus...");
91         order = OrderMock.createOrder1();
92     }
```

Create Mocks for
1. Repository
2. Payment Service

Inject the Mocks into
Order Service

```

@ExtendWith(MockitoExtension.class)

class WarehouseServiceTest {

    @Mock

    private OrderRepository orderRepo;


    @Spy

    private PaymentService paymentService;


    @InjectMocks

    private OrderServiceImpl orderService;

    @DisplayName("1.1 Check All the Paid Orders Ready for Packaging")

    @ParameterizedTest

    @MethodSource("createOrder")

    @Test

    void testForPaidOrders(OrderEntity order) {

        // Given the Order is ready

        // Order is Dynamically shared using the parametrized testing

```

```

// When

    when(orderRepo.saveOrder(order)).thenReturn(order);

    OrderEntity processedOrder =
orderService.processOrder(order);


    // Then Payment Service will return as Payment Accepted

    assertTrue(processedOrder.isCustomerAvailable());

    assertNotNull(processedOrder.getOrderStatus());

    assertEquals(OrderStatus.PAID,
processedOrder.getOrderStatus());

}


// The following method returns an ArrayList of Orders

public ArrayList<OrderEntity> createOrder() {

    return OrderMock.createOrderList();

}

}

```

- @Mock Annotation: Used to declare a mock object. This annotation is typically placed on a field within a test class.

@Mock

```
private MyDependency myDependency;
```

- @InjectMocks Annotation: Used to inject the declared mock objects into the class under test.

@InjectMocks

```
private MyService myService;
```

Spring Boot offers test slices to focus on specific application layers:

- `@WebMvcTest`: For testing the web layer (controllers), auto-configures `MockMvc`.
- `@DataJpaTest`: For testing the data layer (repositories), sets up an in-memory database.
- `@SpringBootTest`: For integration tests requiring the full application context.
- `@MockBean`
 - annotation (for Spring context-aware tests) or `Mockito.mock()` (for pure unit tests) to create mock instances of the dependencies your component relies on.

```

@WebMvcTest
//@SpringBootTest
//@ContextConfiguration(classes=Application.class)

public class EmployeeControllerTest {

    @Autowired

    private MockMvc mockMvc;

    @MockBean

    private EmployeeService employeeService;

    @Test

    public void getAllEmployeeListTest() throws Exception {

        Employee employee[] = new Employee[]{

            Employee.builder().id(1).name("amit kumar").salary(20000).build(),

            Employee.builder().id(2).name("subham kumar").salary(30000).build(),

            Employee.builder().id(3).name("kapil kumar").salary(40000).build(),

        };

        employeeService.addEmployee(employee[0]);

        employeeService.addEmployee(employee[1]);

        employeeService.addEmployee(employee[2]);
    }

```

```

List<Employee> list = Arrays.asList(employee);

Mockito.when(employeeService.getAllEmployeeList()).thenReturn(list);

mockMvc.perform(MockMvcRequestBuilders.get("/emp/employee")).

    andExpect(MockMvcResultMatchers.status().isFound()).

andExpect(MockMvcResultMatchers.contentType(MediaType.APPLICATION_JSON)).

    andExpect(MockMvcResultMatchers.jsonPath("$", Matchers.hasSize(3)))

    .andExpect(MockMvcResultMatchers.jsonPath("$[0].id").value(1)).

    andExpect(MockMvcResultMatchers.jsonPath("$[0].name").value("amit kumar")).

    andExpect(MockMvcResultMatchers.jsonPath("$[0].salary").value(20000));

}

private static ObjectMapper mapper = new ObjectMapper();

@Test

public void addEmployeeTest() throws Exception {

    Employee emp1 = Employee.builder().id(1).name("amit kumar").salary(20000).build();

    Mockito.when(employeeService.addEmployee(emp1)).thenReturn(emp1);

    String json = mapper.writeValueAsString(emp1);

    mockMvc.perform(post("/emp/employee").contentType(MediaType.APPLICATION_JSON).

        content(json).accept(MediaType.APPLICATION_JSON)).

        andExpect(status().isCreated())//.

        .andExpect(jsonPath("$.id").value(1)).

        andExpect(jsonPath("$.name").value("amit kumar")).

        andExpect(jsonPath("$.salary").value(20000));

    }

}

```